

Amazon Cognito



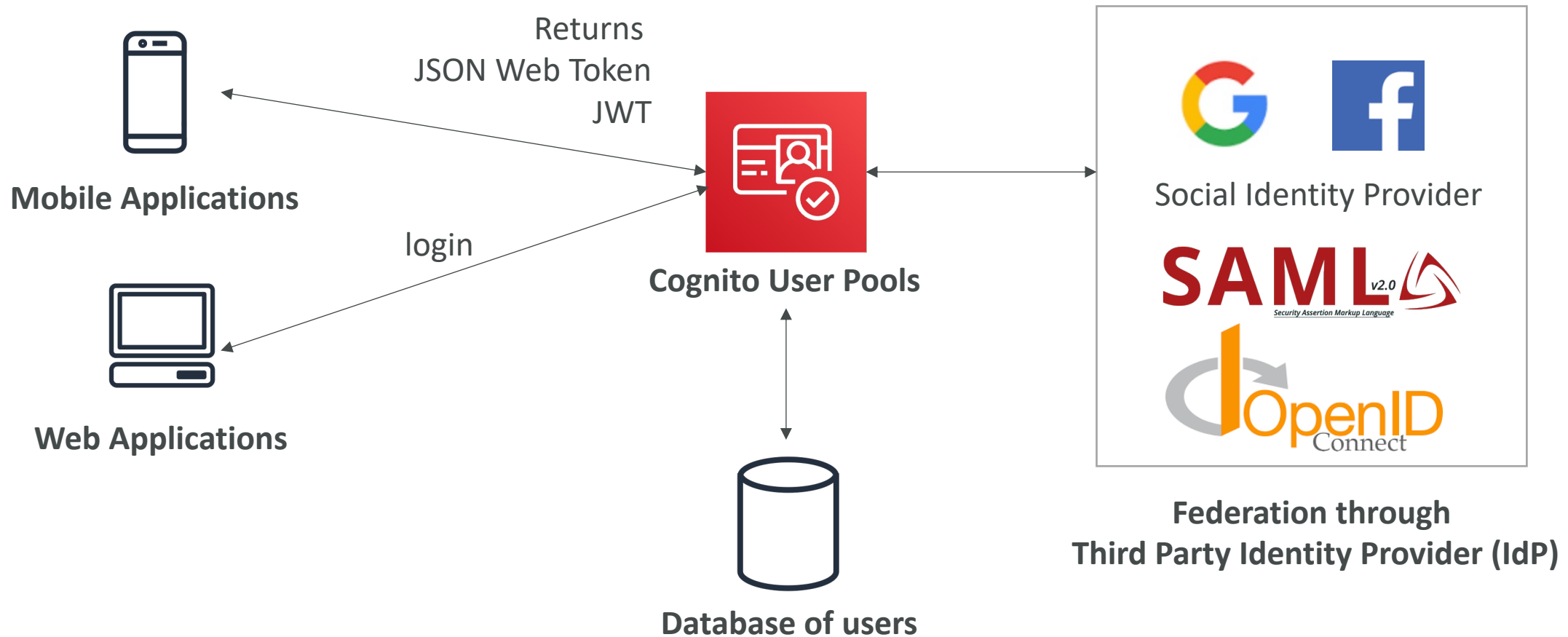
Amazon Cognito

- Give users an identity to interact with our web or mobile application
- **Cognito User Pools:**
 - Sign in functionality for app users
 - Integrate with API Gateway & Application Load Balancer
- **Cognito Identity Pools (Federated Identity):**
 - Provide AWS credentials to users so they can access AWS resources directly
 - Integrate with Cognito User Pools as an identity provider
- **Cognito vs IAM:** “hundreds of users”, “mobile users”, “authenticate with SAML”

Cognito User Pools (CUP) – User Features

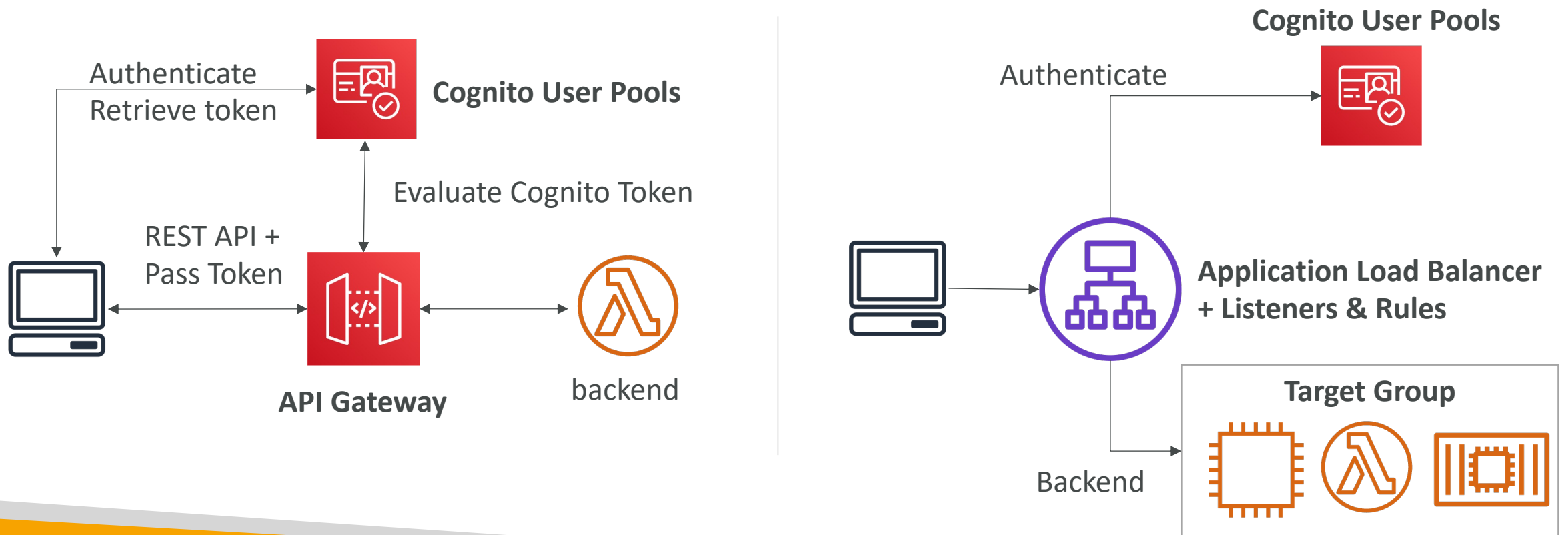
- Create a serverless database of user for your web & mobile apps
- Simple login: Username (or email) / password combination
- Password reset
- Email & Phone Number Verification
- Multi-factor authentication (MFA)
- Federated Identities: users from Facebook, Google, SAML...
- Feature: block users if their credentials are compromised elsewhere
- Login sends back a JSON Web Token (JWT)

Cognito User Pools (CUP) – Diagram



Cognito User Pools (CUP) - Integrations

- CUP integrates with **API Gateway** and **Application Load Balancer**



Cognito User Pools – Lambda Triggers

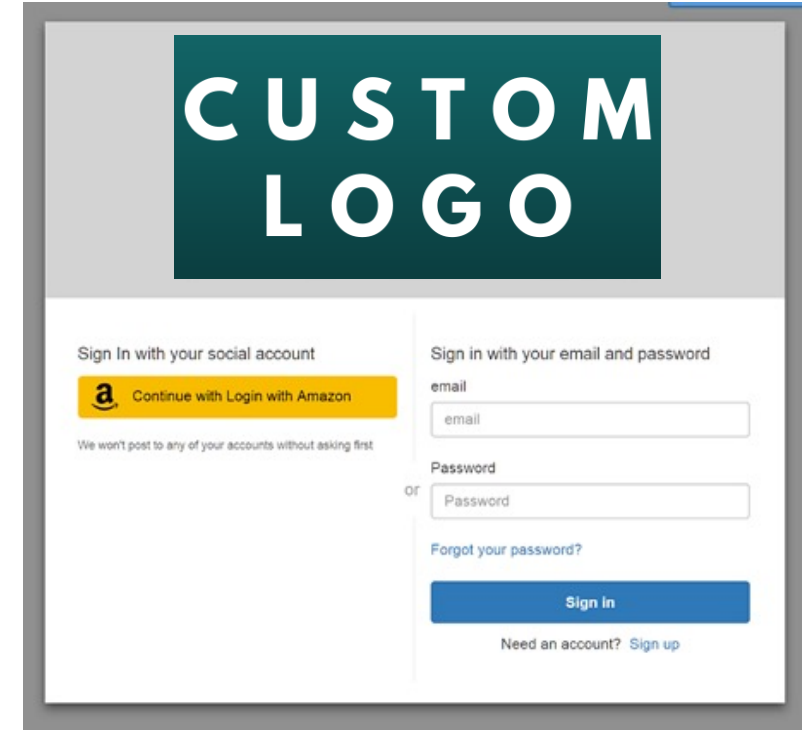
- CUP can invoke a Lambda function synchronously on these triggers:

User Pool Flow	Operation	Description
Authentication Events	Pre Authentication Lambda Trigger	Custom validation to accept or deny the sign-in request
	Post Authentication Lambda Trigger	Event logging for custom analytics
	Pre Token Generation Lambda Trigger	Augment or suppress token claims
Sign-Up	Pre Sign-up Lambda Trigger	Custom validation to accept or deny the sign-up request
	Post Confirmation Lambda Trigger	Custom welcome messages or event logging for custom analytics
	Migrate User Lambda Trigger	Migrate a user from an existing user directory to user pools
Messages	Custom Message Lambda Trigger	Advanced customization and localization of messages
Token Creation	Pre Token Generation Lambda Trigger	Add or remove attributes in Id tokens

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools-working-with-aws-lambda-triggers.html>

Cognito User Pools – Hosted Authentication UI

- Cognito has a **hosted authentication UI** that you can add to your app to handle sign-up and sign-in workflows
- Using the hosted UI, you have a foundation for integration with social logins, OIDC or SAML
- Can customize with a **custom logo** and **custom CSS**



<https://aws.amazon.com/blogs/aws/launch-amazon-cognito-user-pools-general-availability-app-integration-and-federation/>

CUP – Hosted UI Custom Domain

- For custom domains, you must create an ACM certificate in us-east-1
- The custom domain must be defined in the “App Integration” section

[Users](#) | [Groups](#) | [Sign-in experience](#) | [Sign-up experience](#) | [Messaging](#) | [App integration](#) | [User pool properties](#)

Configuration for all app clients

Domain and resource server settings for the user pool. All app clients that enable the Hosted UI use the user pool domain. All app clients can authorize access to user pool resource servers.

Domain [Info](#)

Configure a domain for your Hosted UI and OAuth 2.0 endpoints. You must choose a domain if you need Cognito to create Hosted UI authentication endpoints. If you have an existing domain, you must delete it before assigning a new one.

Actions ▼

Cognito domain

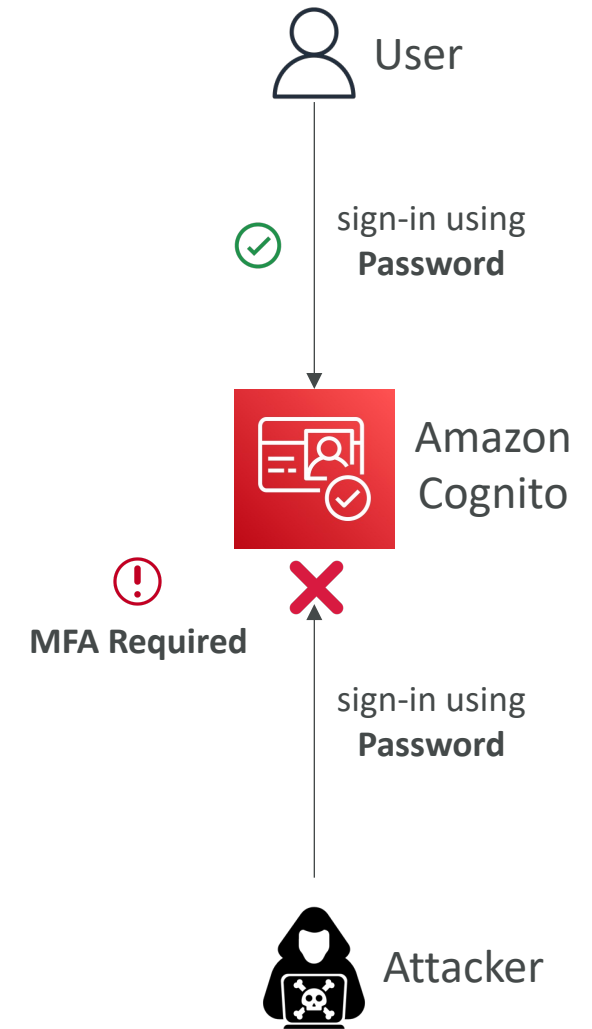
Domain
https://demo-alb.auth.eu-central-1.amazoncognito.com

Custom domain

Domain
-

CUP – Adaptive Authentication

- Block sign-ins or require MFA if the login appears suspicious
- Cognito examines each sign-in attempt and generates a risk score (low, medium, high) for how likely the sign-in request is to be from a malicious attacker
- Users are prompted for a second MFA only when risk is detected
- Risk score is based on different factors such as if the user has used the same device, location, or IP address
- Checks for compromised credentials, account takeover protection, and phone and email verification
- Integration with CloudWatch Logs (sign-in attempts, risk score, failed challenges...)



Decoding a ID Token; JWT – JSON Web Token

- CUP issues JWT tokens (Base64 encoded):
 - Header
 - Payload
 - Signature
- The signature must be verified to ensure the JWT can be trusted
- Libraries can help you verify the validity of JWT tokens issued by Cognito User Pools
- The Payload will contain the user information (sub UUID, given_name, email, phone_number, attributes...)
- From the sub UUID, you can retrieve all users details from Cognito / OIDC

```
<header>.  
{  
  "sub": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee", User ID in Cognito DB  
  "email": "my-test-user@example.com",  
  "email_verified": true,  
  "middle_name": "Jane",  
  "cognito:username": "my-test-user",  
  "cognito:groups": [  
    "my-test-group"  
  ],  
  "cognito:roles": [  
    "arn:aws:iam::111122223333:role/my-test-role"  
  ],  
  "cognito:preferred_role": "arn:aws:iam::111122223333:role/my-test-role",  
  "iss": "https://cognito-idp.us-west-2.amazonaws.com/us-west-2_example",  
  "nonce": "abcdefg",  
  "origin_jti": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",  
  "aud": "xxxxxxxxxxxxexample",  
  "event_id": "64f513be-32db-42b0-b78e-b02127b4f463",  
  "token_use": "id",  
  "auth_time": 1676312777,  
  "exp": 1676316377, Expiry & Issued At  
  "iat": 1676312777,  
  "jti": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",  
}  
.<token signature>
```

ID JWT Token Payload

Application Load Balancer – Authenticate Users

- Your Application Load Balancer can securely authenticate users
 - Offload the work of authenticating users to your load balancer
 - Your applications can focus on their business logic
- Authenticate users through:
 - Identity Provider (IdP): OpenID Connect (OIDC) compliant
 - Cognito User Pools:
 - Social IdPs, such as Amazon, Facebook, or Google
 - Corporate identities using SAML, LDAP, or Microsoft AD
- Must use an HTTPS listener to set authenticate-oidc & authenticate-cognito rules
- OnUnauthenticatedRequest – authenticate (default), deny, allow

Listener details

A listener is a process that checks for connection determine how the load balancer routes request

Protocol

HTTPS ▼

Port

443

1-65535

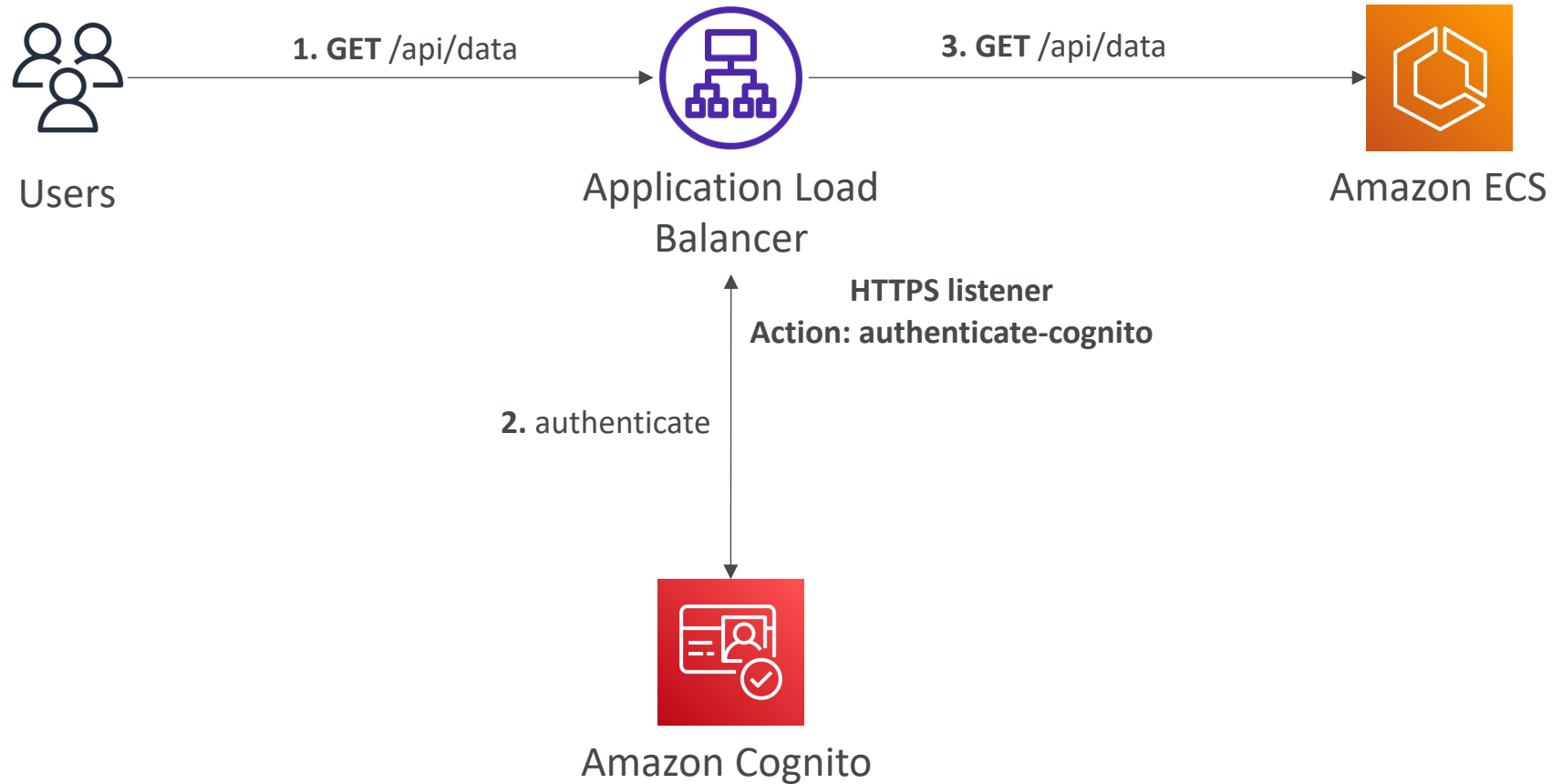
Default actions [Info](#)

Specify the default actions for traffic on this listener. Rules can be configured after the listener is created.

► 1. Authenticate [Info](#)

► 2. Forward to [Info](#)

Application Load Balancer – Cognito Auth.



ALB – Auth through Cognito User Pools

Identity provider - *optional*

Amazon Cognito ▼

Cognito user pool

eu-central-1_N6w7rwX7w

Foobar ▼



[Create user pool](#)

App client

1utbmt28pme63argrj5njpf4u7

OK ▼

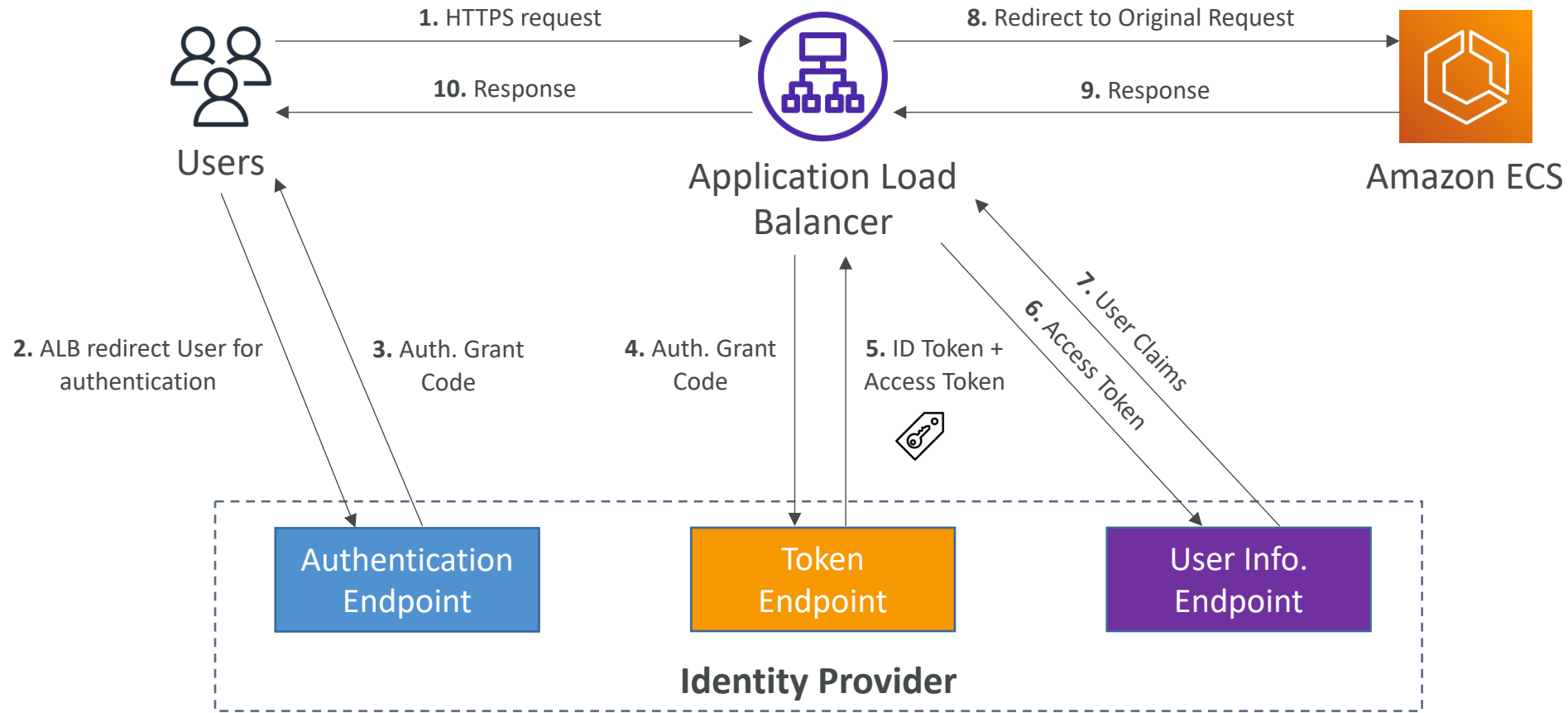


Cognito user pool domain

demo-alb

- Create Cognito User Pool, Client and Domain
- Make sure an ID token is returned
- Add the social or Corporate IdP if needed
- Several URL redirections are necessary
- Allow your Cognito User Pool Domain on your IdP app's callback URL. For example:
 - `https://domain-prefix.auth.region.amazoncognito.com/saml2/idpresponse`
 - `https://user-pool-domain/oauth2/idpresponse`

Application Load Balancer – OIDC Auth.



ALB – Auth. Through an Identity Provider (IdP) That is OpenID Connect (OIDC) Compliant

Identity provider - *optional*

OIDC

Issuer

Enter the OpenID provider.

Authorization endpoint

Enter OpenID provider server endpoint.

Token endpoint

Enter a URL for your token endpoint.

User info endpoint

Enter a URL for your user info endpoint.

Client ID

Enter the client ID.

Client secret

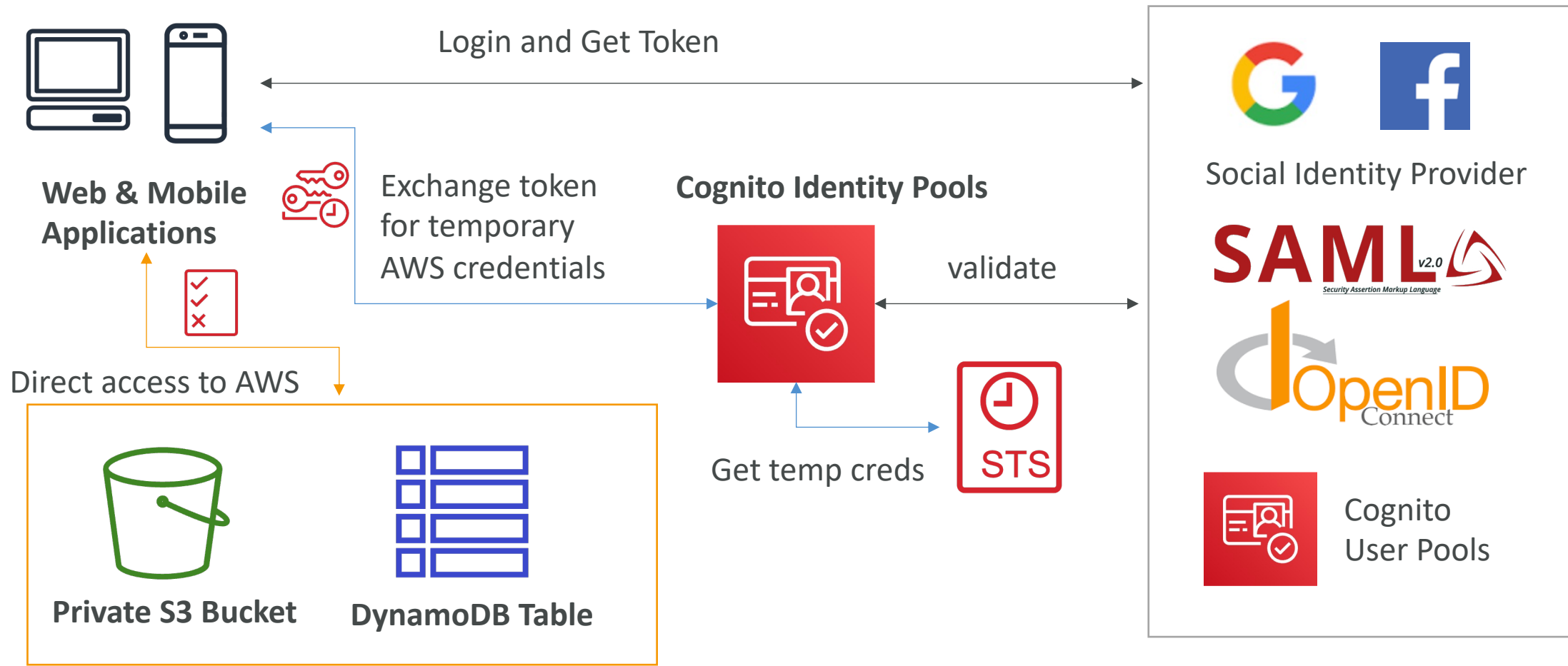
Enter the client secret. Keep track of your client secret. It is required when modifying any rule with an

- Configure a Client ID & Client Secret
- Allow redirect from OIDC to your Application Load Balancer DNS name (AWS provided) and CNAME (DNS Alias of your app)
 - <https://DNS/oauth2/idpresponse>
 - <https://CNAME/oauth2/idpresponse>

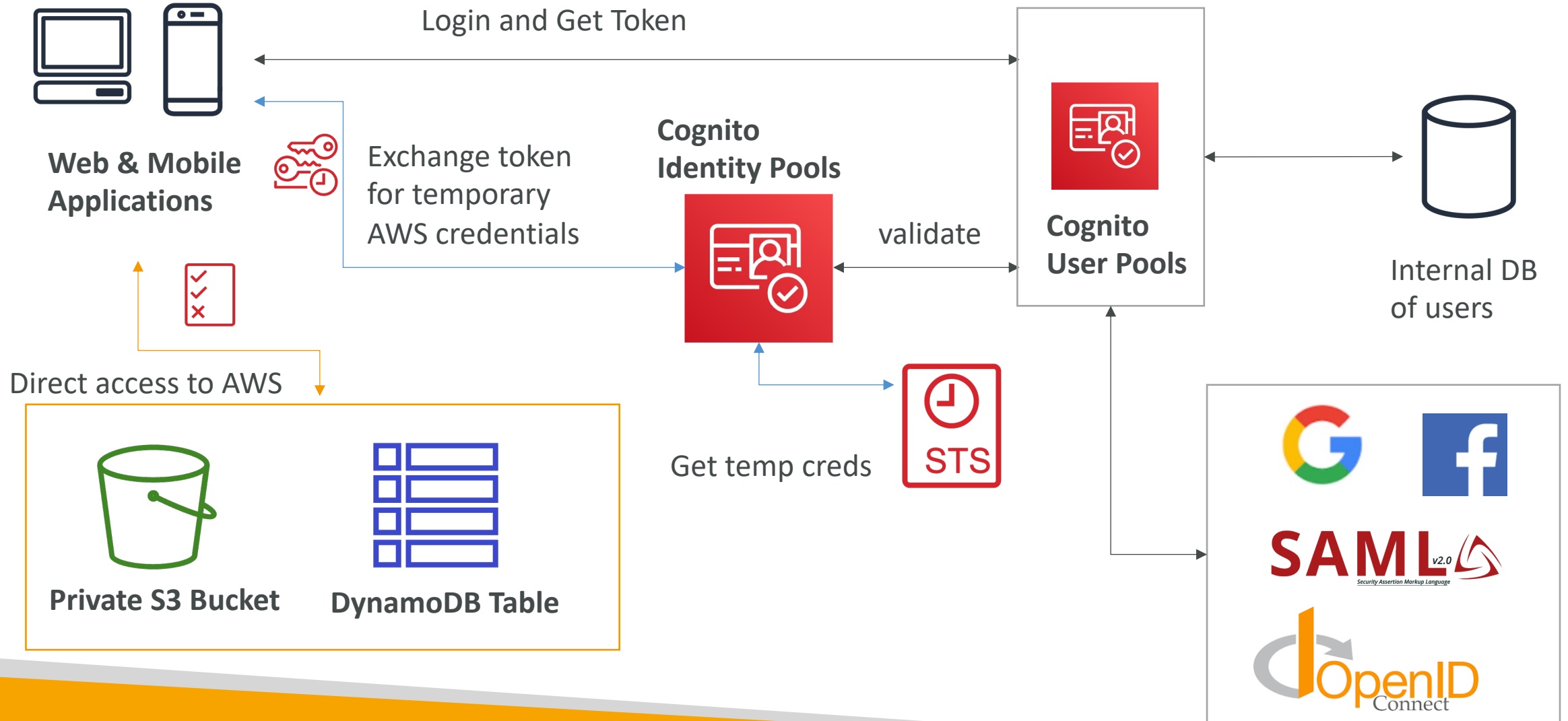
Cognito Identity Pools (Federated Identities)

- Get identities for “users” so they obtain temporary AWS credentials
- Your identity pool (e.g identity source) can include:
 - Public Providers (Login with Amazon, Facebook, Google, Apple)
 - Users in an Amazon Cognito user pool
 - OpenID Connect Providers & SAML Identity Providers
 - Developer Authenticated Identities (custom login server)
 - Cognito Identity Pools allow for unauthenticated (guest) access
- Users can then access AWS services directly or through API Gateway
 - The IAM policies applied to the credentials are defined in Cognito
 - They can be customized based on the user_id for fine grained control

Cognito Identity Pools – Diagram



Cognito Identity Pools – Diagram with CUP



Cognito Identity Pools – IAM Roles

- Default IAM roles for authenticated and guest users
- Define rules to choose the role for each user based on the user's ID
- You can partition your users' access using **policy variables**
- IAM credentials are obtained by Cognito Identity Pools through STS with **AssumeRoleForWebIdentity** STS API Call
- The roles must have a “trust” policy of Cognito Identity Pools

Cognito Identity Pools – Guest User example

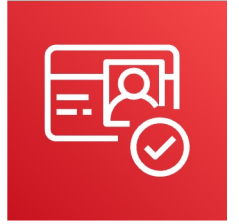
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "s3:GetObject"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:s3:::mybucket/assets/my_picture.jpg"
      ]
    }
  ]
}
```

Cognito Identity Pools – Policy variable on S3

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": ["s3:ListBucket"],
      "Effect": "Allow",
      "Resource": ["arn:aws:s3:::mybucket"],
      "Condition": {"StringLike": {"s3:prefix": ["${cognito-identity.amazonaws.com:sub}/*"]}}
    },
    {
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Effect": "Allow",
      "Resource": ["arn:aws:s3:::mybucket,${cognito-identity.amazonaws.com:sub}/*"]
    }
  ]
}
```

Cognito Identity Pools – DynamoDB

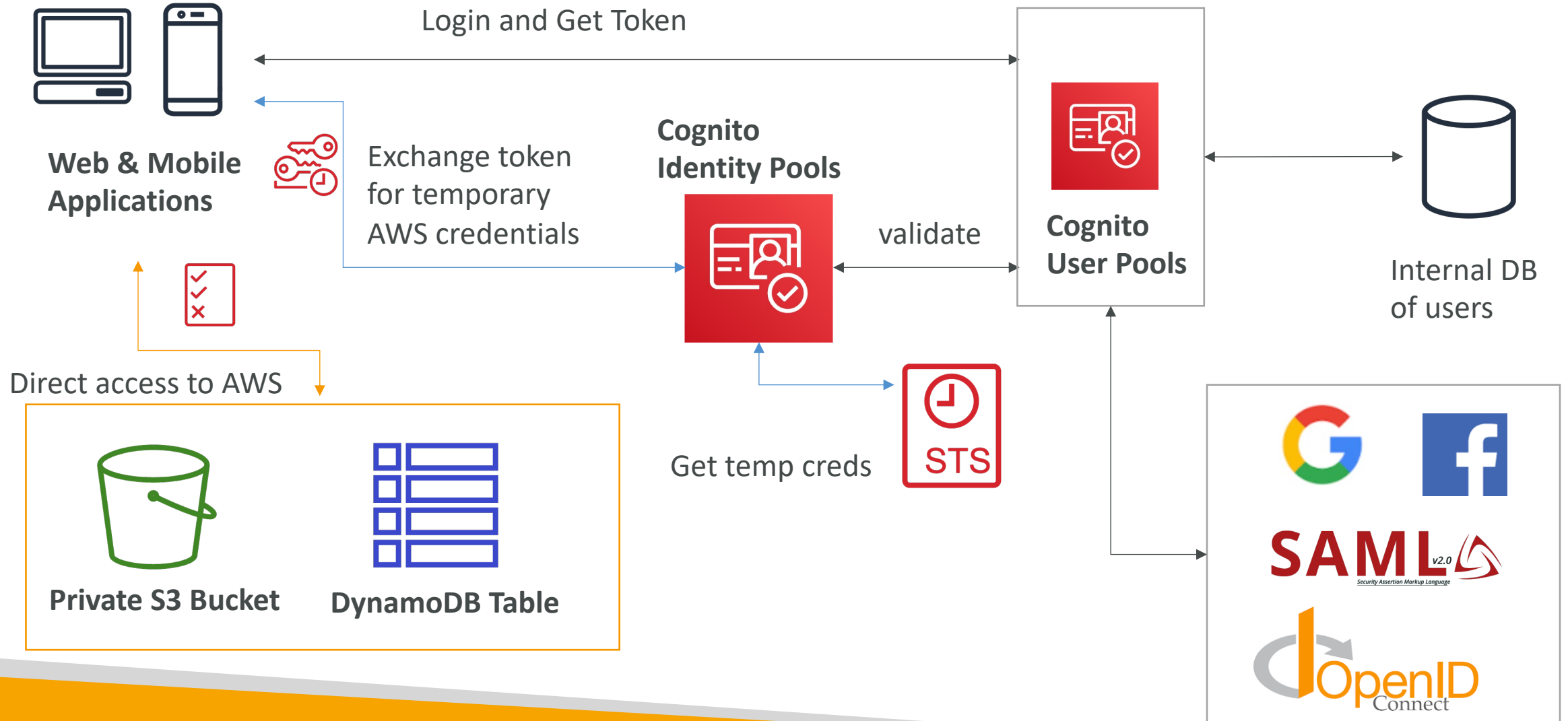
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem", "dynamodb:BatchGetItem", "dynamodb:Query",
        "dynamodb:PutItem", "dynamodb:UpdateItem", "dynamodb>DeleteItem",
        "dynamodb:BatchWriteItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-west-2:123456789012:table/MyTable"
      ],
      "Condition": {
        "ForAllValues:StringEquals": {
          "dynamodb:LeadingKeys": [
            "${cognito-identity.amazonaws.com:sub}"
          ]
        }
      }
    }
  ]
}
```



Cognito User Pools vs Identity Pools

- **Cognito User Pools (for authentication = identity verification)**
 - Database of users for your web and mobile application
 - Allows to federate logins through Public Social, OIDC, SAML...
 - Can customize the hosted UI for authentication (including the logo)
 - Has triggers with AWS Lambda during the authentication flow
 - Adapt the sign-in experience to different risk levels (MFA, adaptive authentication, etc...)
- **Cognito Identity Pools (for authorization = access control)**
 - Obtain AWS credentials for your users
 - Users can login through Public Social, OIDC, SAML & Cognito User Pools
 - Users can be unauthenticated (guests)
 - Users are mapped to IAM roles & policies, can leverage policy variables
- **CUP + CIP = authentication + authorization**

Cognito Identity Pools – Diagram with CUP

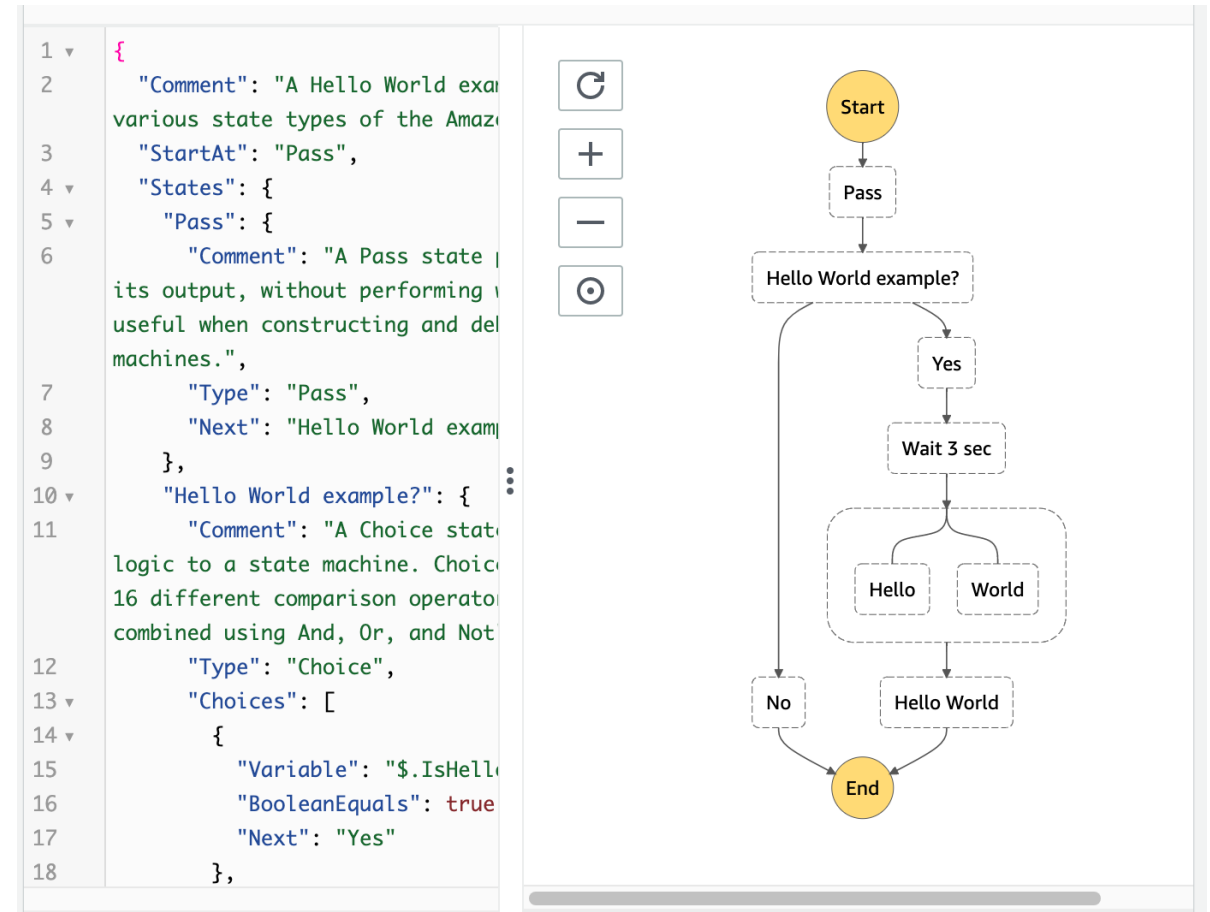


Other Serverless



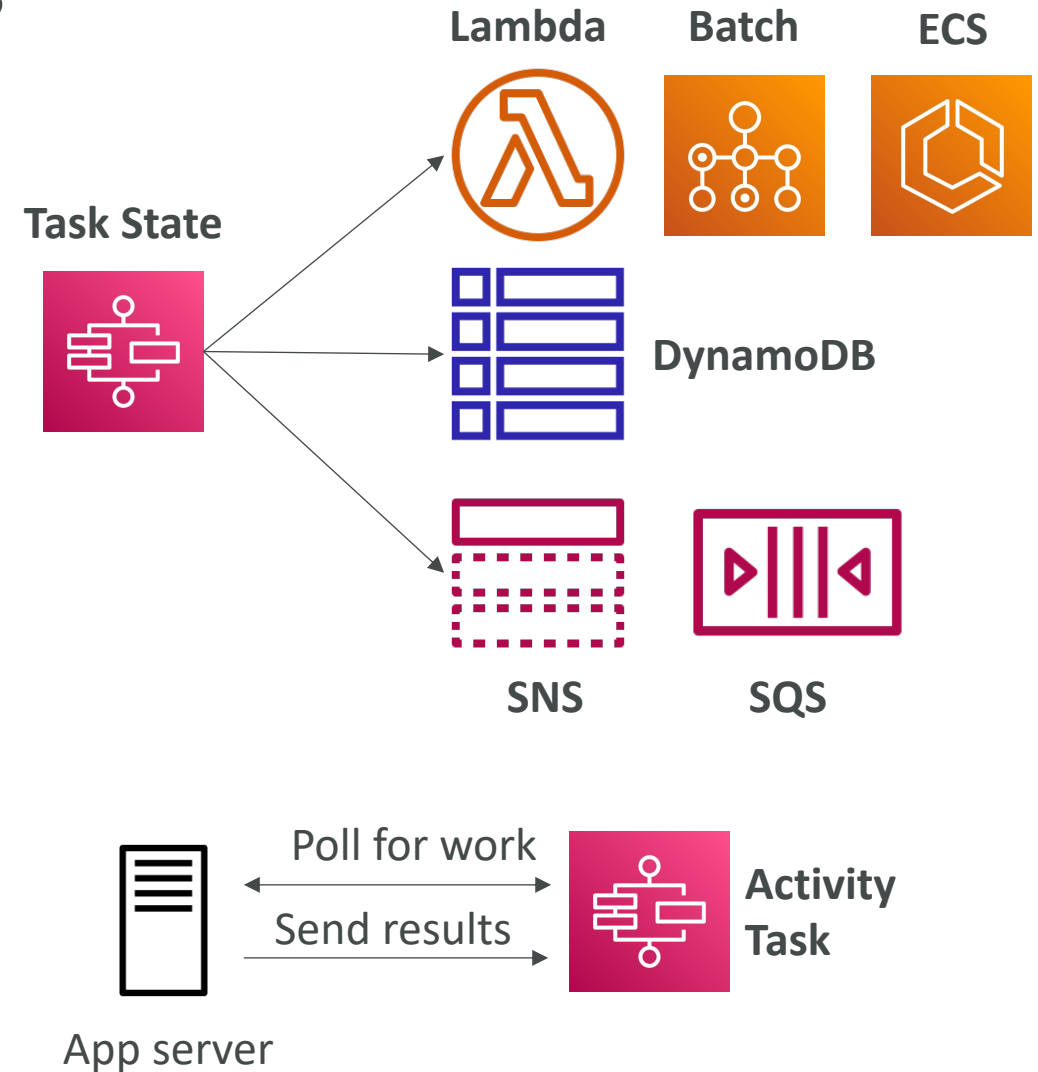
AWS Step Functions

- Model your **workflows as state machines (one per workflow)**
 - Order fulfillment, Data processing
 - Web applications, Any workflow
- Written in JSON
- Visualization of the workflow and the execution of the workflow, as well as history
- Start workflow with SDK call, API Gateway, Event Bridge (CloudWatch Event)



Step Function – Task States

- Do some work in your state machine
- Invoke one AWS service
 - Can invoke a **Lambda** function
 - Run an AWS Batch job
 - Run an ECS task and wait for it to complete
 - Insert an item from DynamoDB
 - Publish message to SNS, SQS
 - Launch another Step Function workflow...
- Run an one Activity
 - EC2, Amazon ECS, on-premises
 - Activities poll the Step functions for work
 - Activities send results back to Step Functions



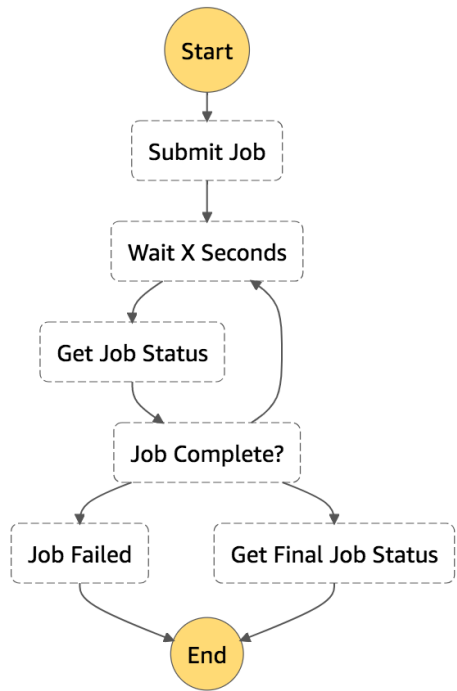
Example – Invoke Lambda Function

```
"Invoke Lambda function": {  
  "Type": "Task",  
  "Resource": "arn:aws:states:::lambda:invoke",  
  "Parameters": {  
    "FunctionName":  
      "arn:aws:lambda:REGION:ACCOUNT_ID:function:FUNCTION_NAME",  
    "Payload": {  
      "Input.$": "$"  
    }  
  },  
  "Next": "NEXT_STATE",  
  "TimeoutSeconds": 300  
}
```

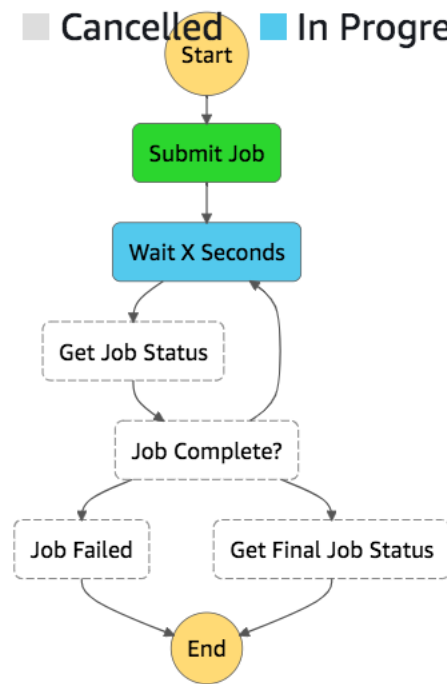
Step Function - States

- **Choice State** - Test for a condition to send to a branch (or default branch)
- **Fail or Succeed State** - Stop execution with failure or success
- **Pass State** - Simply pass its input to its output or inject some fixed data, without performing work.
- **Wait State** - Provide a delay for a certain amount of time or until a specified time/date.
- **Map State** - Dynamically iterate steps.'
- *Parallel State* - *Begin parallel branches of execution.*

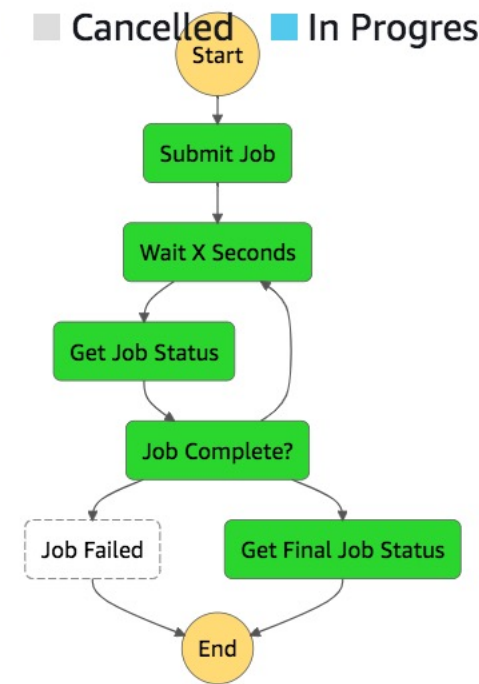
Visual workflow in Step Functions



■ Success ■ Failed ■ Cancelled ■ In Progress



■ Success ■ Failed ■ Cancelled ■ In Progress



Error Handling in Step Functions

- Any state can encounter runtime errors for various reasons:
 - State machine definition issues (for example, no matching rule in a Choice state)
 - Task failures (for example, an exception in a Lambda function)
 - Transient issues (for example, network partition events)
- Use **Retry** (to retry failed state) and **Catch** (transition to failure path) in the State Machine to handle the errors instead of inside the Application Code
- Predefined error codes:
 - States.ALL : matches any error name
 - States.Timeout: Task ran longer than TimeoutSeconds or no heartbeat received
 - States.TaskFailed: execution failure
 - States.Permissions: insufficient privileges to execute code
- The state may report its own errors

Step Functions – Retry (Task or Parallel State)

```
"HelloWorld": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:FUNCTION_NAME",
  "Retry": [
    {
      "ErrorEquals": ["CustomError"],
      "IntervalSeconds": 1,
      "MaxAttempts": 2,
      "BackoffRate": 2.0
    },
    {
      "ErrorEquals": ["States.TaskFailed"],
      "IntervalSeconds": 30,
      "MaxAttempts": 2,
      "BackoffRate": 2.0
    },
    {
      "ErrorEquals": ["States.ALL"],
      "IntervalSeconds": 5,
      "MaxAttempts": 5,
      "BackoffRate": 2.0
    }
  ],
  "End": true
}
```

- Evaluated from top to bottom
- **ErrorEquals**: match a specific kind of error
- **IntervalSeconds**: initial delay before retrying
- **BackoffRate**: multiple the delay after each retry
- **MaxAttempts**: default to 3, set to 0 for never retried
- When max attempts are reached, the **Catch** kicks in

Step Functions – Catch (Task or Parallel State)

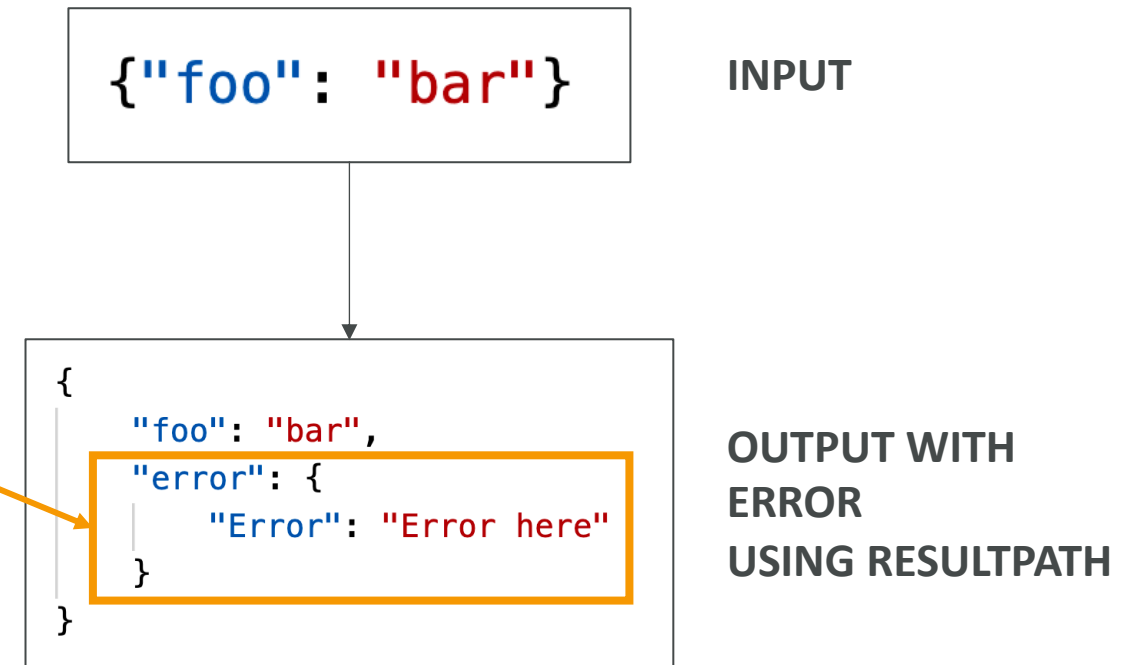
```
"HelloWorld": {  
  "Type": "Task",  
  "Resource": "arn:aws:lambda:....",  
  "Catch": [  
    {  
      "ErrorEquals": ["CustomError"],  
      "Next": "CustomErrorFallback"  
    },  
    {  
      "ErrorEquals": ["States.TaskFailed"],  
      "Next": "ReservedTypeFallback"  
    },  
    {  
      "ErrorEquals": ["States.ALL"],  
      "Next": "NextTask",  
      "ResultPath": "$.error"  
    }  
  ],  
  "End": true  
},  
"CustomErrorFallback": {  
  "Type": "Pass",  
  "Result": "This is a fallback from a custom lambda function exception"  
  "End": true  
},
```

- Evaluated from top to bottom
- **ErrorEquals**: match a specific kind of error
- **Next**: State to send to
- **ResultPath** - A path that determines what input is sent to the state specified in the Next field.

Step Function – ResultPath

- Include the error in the input

```
"HelloWorld": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:....",
  "Catch": [{
    "ErrorEquals": ["States.ALL"],
    "Next": "NextTask",
    "ResultPath": "$.error"
  }],
  "End": true
},
"NextTask": {
  "Type": "Pass",
  "Result": "This is a fallback from a reserved error code",
  "End": true
}
```



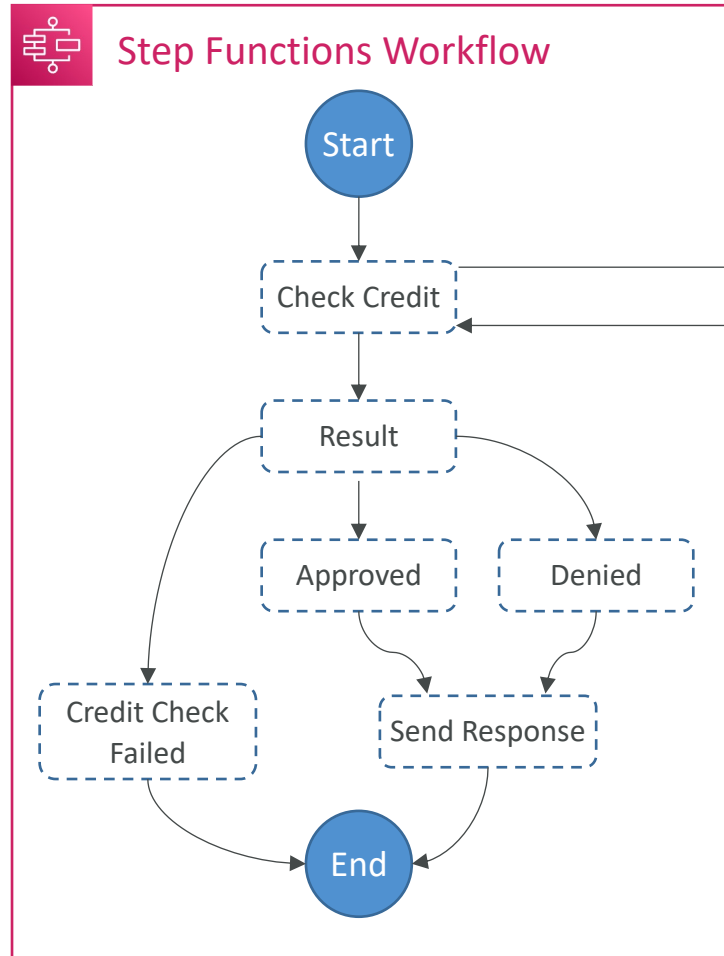
Step Functions – Wait for Task Token

- Allows you to pause Step Functions during a Task until a Task Token is returned
- Task might wait for other AWS services, human approval, 3rd party integration, call legacy systems...
- Append `.waitForTaskToken` to the **Resource** field to tell Step Functions to wait for the Task Token to be returned

```
"Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken"
```

- Task will pause until it receives that Task Token back with a **SendTaskSuccess** or **SendTaskFailure** API call

Step Functions – Wait for Task Token



```
{  
  "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",  
  "Parameters": {  
    "QueueUrl": "https://sqs.eu-west-1.amazonaws.com/123456789012/MyQueue",  
    "MessageBody": {  
      "Input.$": "$",  
      "TaskToken.$": "$$.Task.Token"  
    }  
  }  
}
```

call SQS with Task Token

```
{  
  "output": "CreditAccepted",  
  "taskToken": "83p1E0dMccoxLzFhgLsdKzpK9mBVkZsp7d9yrT1W"  
}
```

Task completed
SendTaskSuccess API call

SQS Queue
(MyQueue)



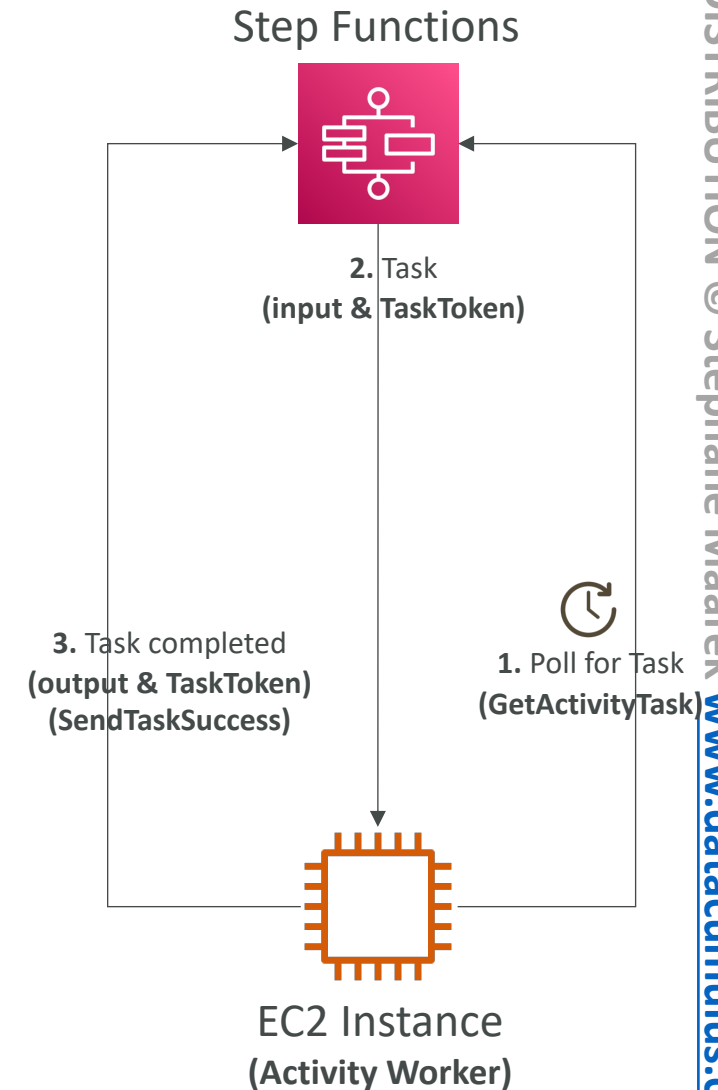
poll messages



Process Messages

Step Functions – Activity Tasks

- Enables you to have the Task work performed by an *Activity Worker*
- **Activity Worker** apps can be running on EC2, Lambda, mobile device...
- Activity Worker poll for a Task using **GetActivityTask** API
- After Activity Worker completes its work, it sends a response of its success/failure using **SendTaskSuccess** or **SendTaskFailure**
- To keep the Task active:
 - Configure how long a task can wait by setting **TimeoutSeconds**
 - Periodically send a heartbeat from your Activity Worker using **SendTaskHeartBeat** within the time you set in **HeartBeatSeconds**
- By configuring a long **TimeoutSeconds** and actively sending a heartbeat, Activity Task can wait up to 1 year



Step Functions – Standard vs. Express

Max. Duration

Up to 1 year

Execution Model

Exactly-once Execution

Execution Rate

Over 2000 / second

Execution History

Up to 90 days or using CloudWatch

Pricing

of State Transitions

Use cases

Non-idempotent actions
(e.g., processing Payments)

Step Functions
Workflows

Standard (default)

Express

Asynchronous
At-least once

Synchronous
At-most once

Doesn't wait for Workflow to complete
(get results from CW Logs)

You don't need an immediate response
(e.g., messaging services)

Must manage idempotence

Wait for Workflow to complete

You need an immediate response
(e.g., orchestrate microservices)

Can be invoked from
API Gateway or Lambda function

Up to 5 minutes

Over 100,000 / second

CloudWatch Logs

of executions, duration,
and memory consumption

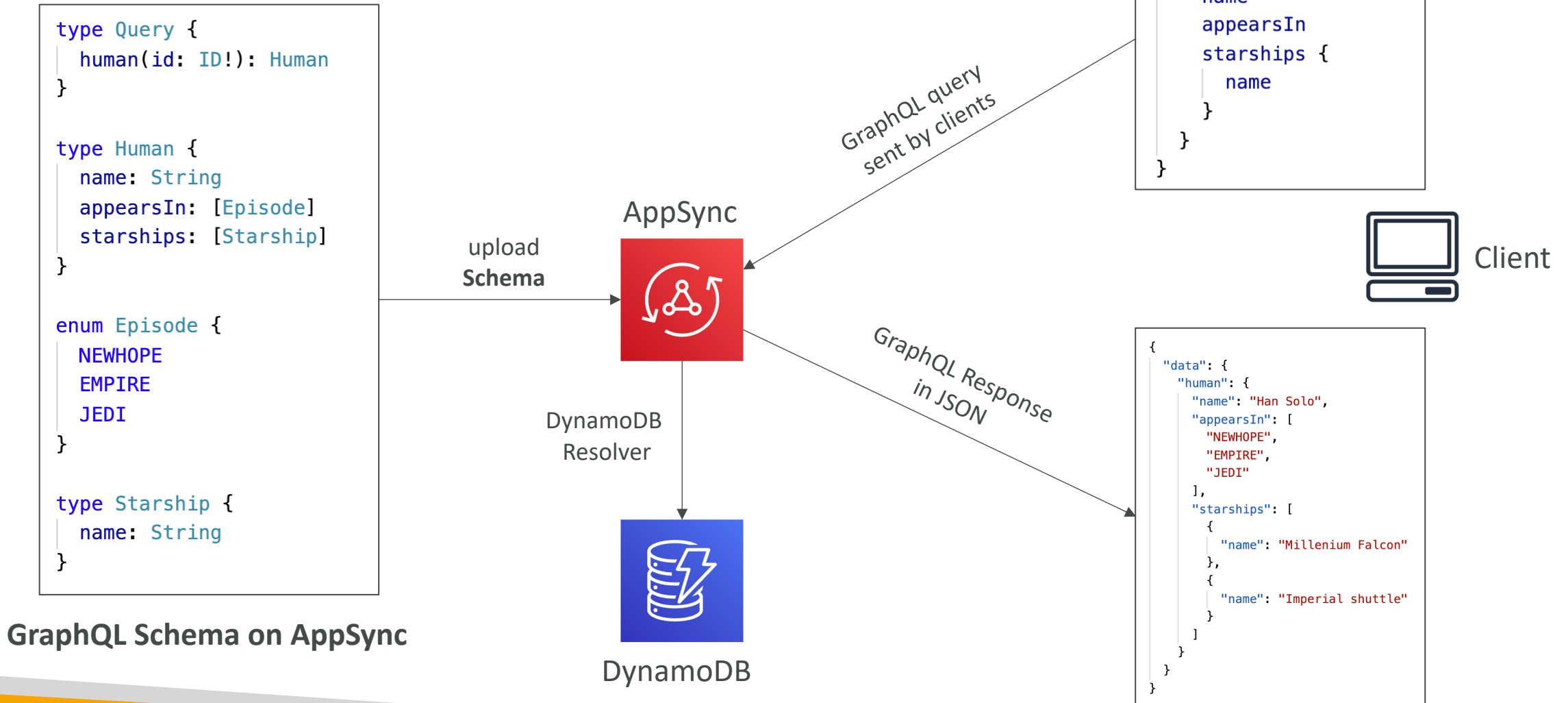
IoT data ingestion, streaming data,
mobile app backends, ...



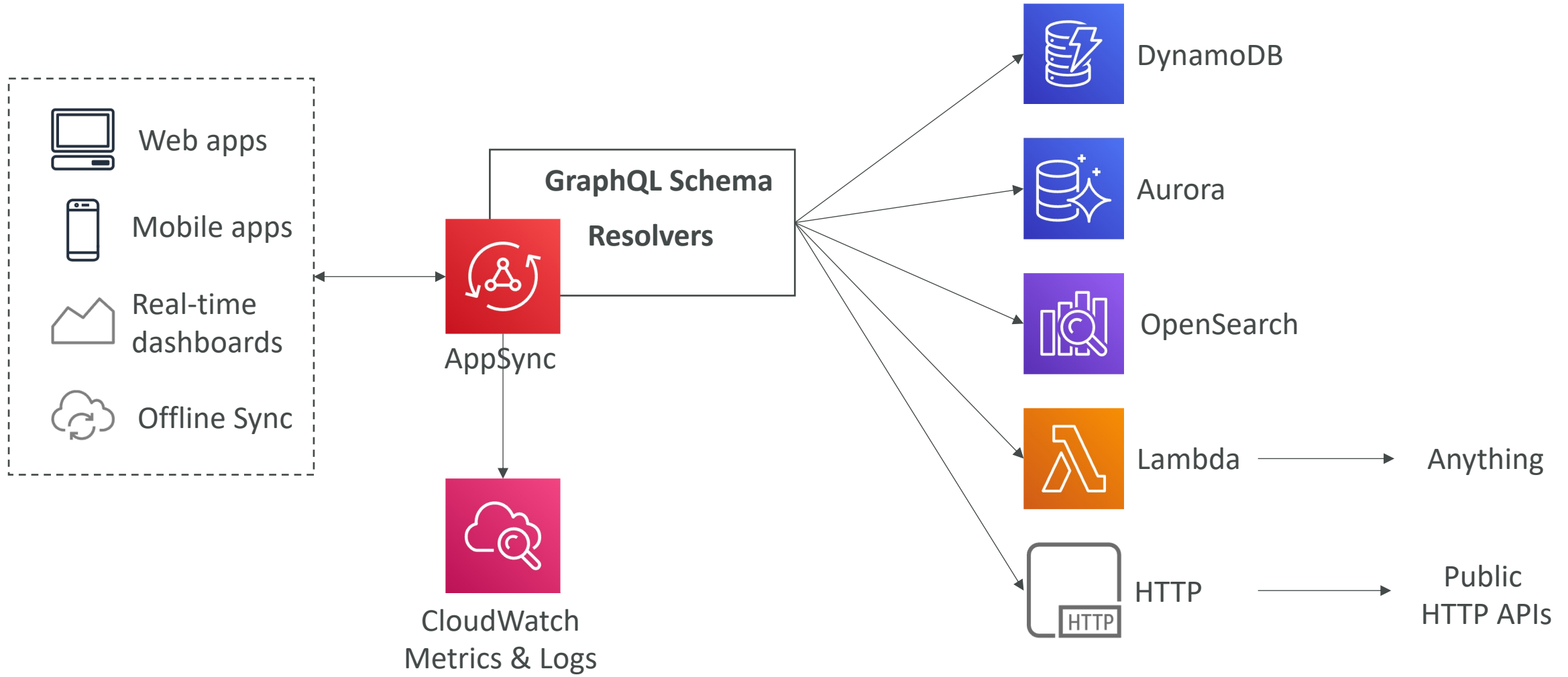
AWS AppSync - Overview

- AppSync is a managed service that uses GraphQL
- GraphQL makes it easy for applications to get exactly the data they need.
- This includes combining data from **one or more sources**
 - NoSQL data stores, Relational databases, HTTP APIs...
 - Integrates with DynamoDB, Aurora, OpenSearch & others
 - Custom sources with AWS Lambda
- Retrieve data in **real-time with WebSocket** or MQTT on WebSocket
- For mobile apps: local data access & data synchronization
- It all starts with uploading one GraphQL schema

GraphQL Example



AppSync Diagram



AppSync – Security

- There are four ways you can authorize applications to interact with your AWS AppSync GraphQL API:
- **API_KEY**
- **AWS_IAM**: IAM users / roles / cross-account access
- **OPENID_CONNECT**: OpenID Connect provider / JSON Web Token
- **AMAZON_COGNITO_USER_POOLS**
- For custom domain & HTTPS, use CloudFront in front of AppSync

AWS Amplify

Create mobile and web applications



Amplify Studio

Visually build a full-stack app,
both front-end UI and a backend.



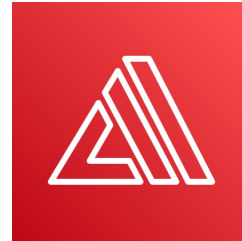
Amplify CLI

Configure an Amplify backend
With a guided CLI workflow



Amplify Libraries

Connect your app to existing AWS
Services (Cognito, S3 and more)

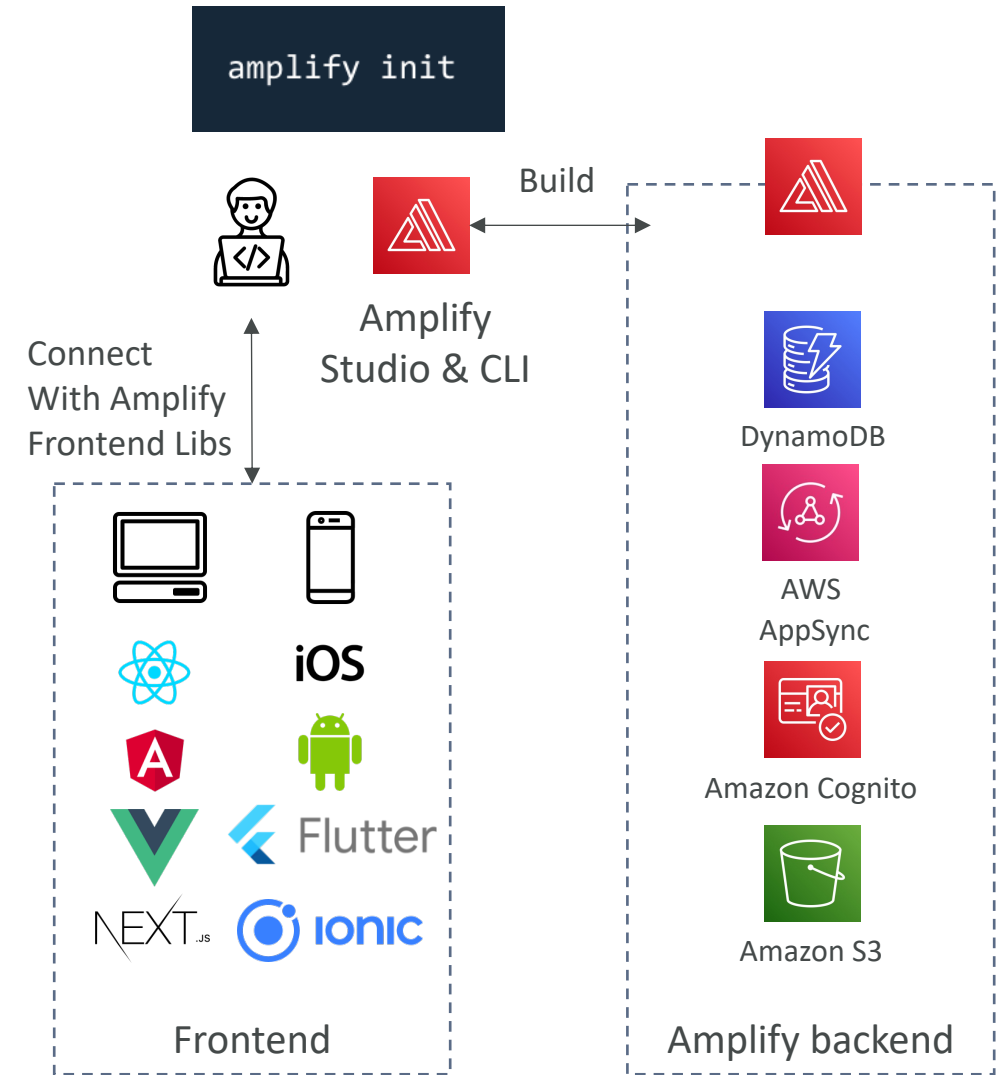


Amplify Hosting

Host secure, reliable, fast web apps or websites
via the AWS content delivery network.

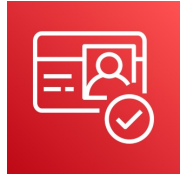
AWS Amplify

- Set of tools to get started with creating **mobile and web applications**
- “Elastic Beanstalk for mobile and web applications”
- Must-have features such as **data storage, authentication, storage, and machine-learning**, all powered by AWS services
- **Front-end libraries** with ready-to-use components for React.js, Vue, Javascript, iOS, Android, Flutter, etc...
- Incorporates AWS best practices to for reliability, security, scalability
- Build and deploy with the **Amplify CLI** or **Amplify Studio**



AWS Amplify – Important Features

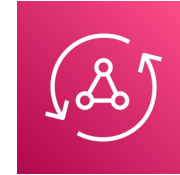
```
amplify add auth
```



AUTHENTICATION

- Leverages Amazon Cognito
- User registration, authentication, account recovery & other operations
- Support MFA, Social Sign-in, etc...
- Pre-built UI components
- Fine-grained authorization

```
amplify add api
```

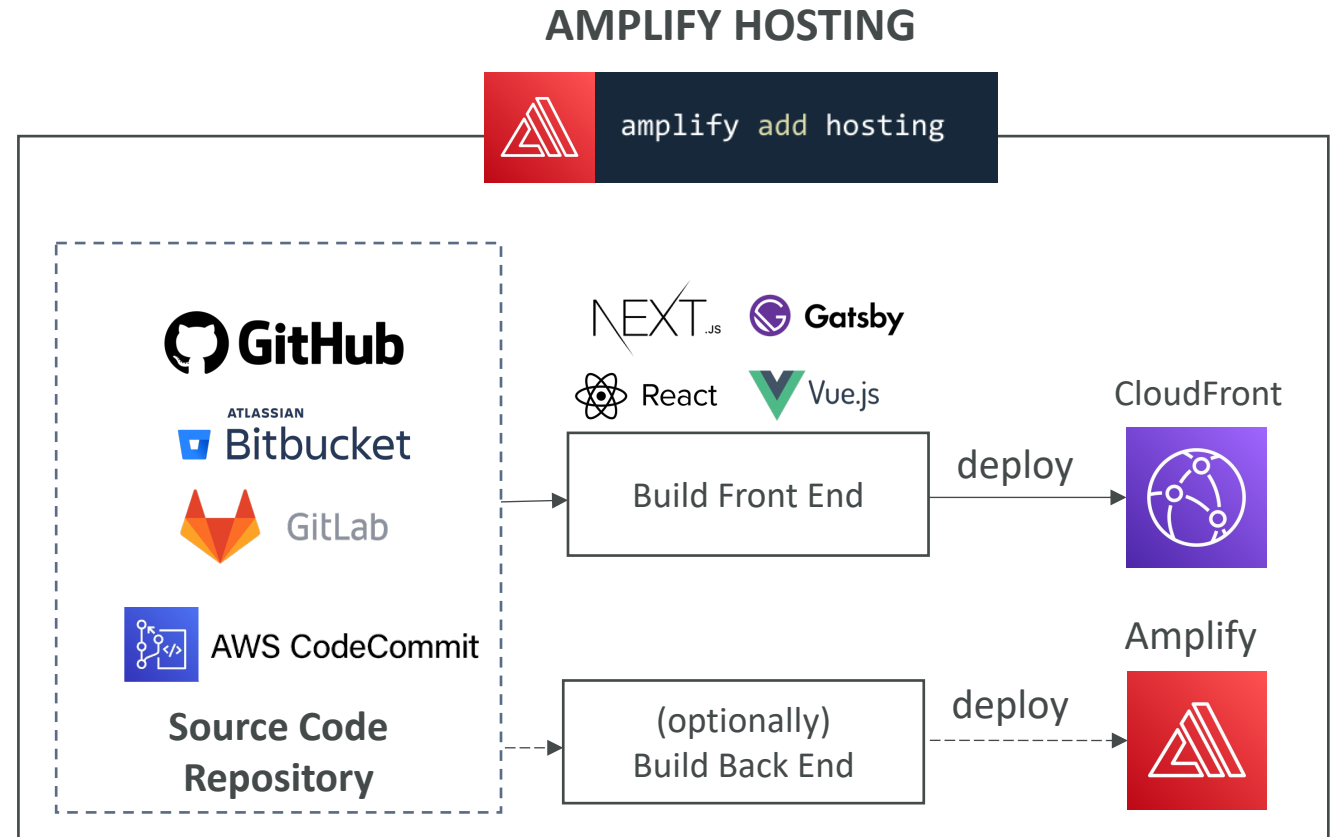


DATASTORE

- Leverages Amazon AppSync and Amazon DynamoDB
- Work with local data and have **automatic synchronization to the cloud** without complex code
- Powered by GraphQL
- Offline and real-time capabilities
- Visual data modeling w/ Amplify Studio

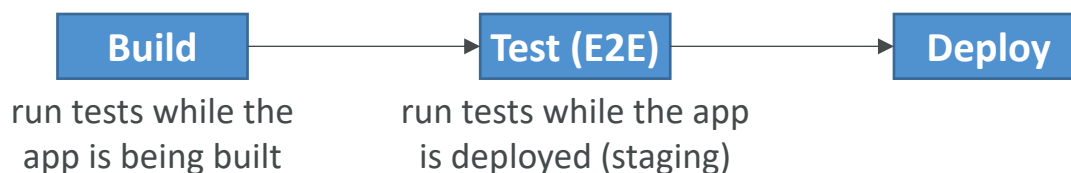
AWS Amplify Hosting

- Build and Host Modern Web Apps
- CI/CD (build, test, deploy)
- Pull Request Previews
- Custom Domains
- Monitoring
- Redirect and Custom Headers
- Password protection



AWS Amplify – End-to-End (E2E) Testing

- Run end-to-end (E2E) tests in the **test phase** in Amplify
- Catch regressions before pushing code to production
- Use the test step to run any test commands at build time (**amplify.yml**)
- Integrated with Cypress testing framework
 - Allows you to generate UI report for your tests



```
test:
  phases:
    preTest:
      commands:
        - npm ci
        - npm install -g pm2
        - npm install -g wait-on
        - npm install mocha mochawesome ...
        - pm2 start npm -- start
        - wait-on http://localhost:3000
    test:
      commands:
        - 'npx cypress run --reporter ...'
    postTest:
      commands:
        - npx mochawesome-merge cypress/...
        - pm2 kill

artifacts:
  baseDirectory: cypress
  configFile: "**/mochawesome.json"
  files:
    - "**/*.png"
    - "**/*.mp4"
```

amplify.yml

Advanced Identity in AWS